Open in Colab

Bitwise Arithmetic

In computere science, `0` represents logical false while `1` represents logical true. So we can do boolean arithmetic on `0` and `1`.

a	b	a&b	a|b	a^b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Here

- `&` is called *bitwise and*
- `|` is called *bitwise or*
- `^` is called *bitwise xor*

In [36]:
```
1 & 0 ^ 1 | 0
```

Out[36]: 1

For integers other than `0` and `1`, bitwise operations are defined on their binary representation.

Binary Representation

Writing a number in base 2 is called *binary* representation. Binary numbers starts with prefix `0b` in Python.

In [37]:
```
0b111000011011
```

Out[37]: 3611

To see the binary representaion of a number use builtin function `bin()`

In [38]:
```
bin(3611)
```

Out[38]: '0b111000011011'

Since each integer has a binary representation we can generalize this operators to any integer like the following example:

In [39]:
```
(     0b1101011001
    & 0b1010000001
#   --------------
==    0b1000000001 )
```

Out[39]: True

In [40]:
```
(     0b1101011001
    | 0b1010000001
```

```
#   --------------
==    0b1111011001 )
```

Out[40]: True

In [41]:
```
(     0b1101011001
    ^ 0b1010000001
#   --------------
==    0b0111011000 )
```

Out[41]: True

Bitwise left shift adds given number of binary `0` s to the end of the number. This means multiplication by a power of `2` .

In [42]:
```
# shift left by 1 is equivalent to multiplication by 2
bin(0b1101011001 << 1)
```

Out[42]: '0b11010110010'

Bitwise right shift removes given number of digits from the end of the number. This means floor division by a power of `2` .

In [43]:
```
# Shift right by 3 is equivalent to floor division by 8
bin(0b1101011001 >> 3)
```

Out[43]: '0b1101011'

Operator Precedence of Bitwise Operators

The arithmetic operators have higher precedence than the bitwise operators. The precedence of bitwise operators are as follows:

Operator	Description
`<<` `>>`	Shift operators
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR

In [44]:
```
2*3+5 << 3 | 1 & 50
```

Out[44]: 88

is equivalent to:

In [45]:
```
((((2*3)+5) << 3) | (1 & 50))
```

Out[45]: 88

Hexadecimal Representatin

Writing a number in base 16 is called *hexadecimal* representation. Hexadecimal representatin is more compact form of binary representatin.

To write the digits in base 16 use:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Any hexadecimal number starts with prefix `0x` and the number is case insensitive. This means `0x1A` is equal to `0x1a`.

In [46]:
```
0xFE
```

Out[46]:
```
254
```

In [47]:
```
0x12ACD00002
```

Out[47]:
```
80208723970
```

Octal Representation

Writing a number in base 8 is called *octal* representation.

In [48]:
```
0o76543210
```

Out[48]:
```
16434824
```

In [49]:
```
oct(16434824)
```

Out[49]:
```
'0o76543210'
```